

Лекция 8

1. Прогон программы синхронизации с помощью переменной замка

```
#include <windows.h>
#include <stdio.h>
#include <math.h>
int Sum = 0, iNumber = 5, jNumber = 300000;
DWORD WINAPI SecondThread(LPVOID) {
    int i, j;
    double a, b = 1.;
    for (i = 0; i < iNumber; i++)
    {
        for (j = 0; j < jNumber; j++)
        {
            Sum = Sum + 1; a = sin(b);
        }
    }
    return 0;
}
void main() {
    int i, j;
    double a, b = 1.;
    HANDLE hThread;
    DWORD IDThread;
    hThread = CreateThread(NULL, 0, SecondThread, NULL, 0, &IDThread);
    if (hThread == NULL) return;
    for (i = 0; i < iNumber; i++)
    {
        for (j = 0; j < jNumber; j++)
        {
            Sum = Sum - 1; a = sin(b);
        }
        printf(" %d ", Sum);
    }
    WaitForSingleObject(hThread, INFINITE);
    printf(" %d ", Sum);
}
```

```
-13611 373159 327947 308497 229275 229275
C:\Users\Анастасия\source\repos\lecture8\x64\Debug\le
Нажмите любую клавишу, чтобы закрыть это окно...
```

2. Семафоры

```

#include <windows.h>
#include <stdio.h>
#include <math.h>
int Sum = 0, iNumber = 5, jNumber = 300000;
HANDLE hFirstSemaphore, hSecondSemaphore;
DWORD WINAPI SecondThread(LPVOID)
{
    int i, j;
    double a, b = 1.;
    for (i = 0; i < iNumber; i++)
    {
        WaitForSingleObject(hFirstSemaphore, INFINITE);
        for (j = 0; j < jNumber; j++)
        {
            Sum = Sum + 1; a = sin(b);
        }
        ReleaseSemaphore(hFirstSemaphore, 1, NULL);
    }
    return 0;
}
int main()
{
    int i, j;
    HANDLE hThread;
    DWORD IDThread;
    double a, b = 1.;
    hFirstSemaphore = CreateSemaphore(NULL, 0, 1, L"MyFirstSemaphore");
    hSecondSemaphore = CreateSemaphore(NULL, 1, 1, L"MySecondSemaphore1");
    hThread = CreateThread(NULL, 0, SecondThread, NULL, 0, &IDThread);
    if (hThread == NULL) return 1;
    for (i = 0; i < iNumber; i++)
    {

```

```

        WaitForSingleObject(hSecondSemaphore, INFINITE);
        for (j = 0; j < jNumber; j++)
        {
            Sum = Sum - 1; a = sin(b);
        }
        printf(" %d ", Sum);
        ReleaseSemaphore(hSecondSemaphore, 1, NULL);
    }
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hFirstSemaphore);
    CloseHandle(hSecondSemaphore);
    printf(" %d ", Sum);
    return 0;
}

```

```

-300000 -600000 -900000 -1200000 -1500000

```

Лекция 9 не работает, как нужно

3. Прогон программы выделения памяти при помощи функции Virtual Alloc

```

#include <windows.h>
#include <stdio.h>
int main(int)
{
    PVOID pMem = NULL;
    char* String;
    char* pMemCommitted;
    int nPageSize = 4096;
    int Shift = 4080;
    pMem = VirtualAlloc(0, nPageSize * 16, MEM_RESERVE,
        PAGE_READWRITE);
    pMemCommitted = (char*)pMem + 3 * nPageSize;
    VirtualAlloc(pMemCommitted, nPageSize, MEM_COMMIT,
        PAGE_READWRITE);
    String = pMemCommitted + Shift;
    sprintf_s(String, 100, "Hello, world");
    printf("%s\n", String);
    VirtualFree(String, 0, MEM_RELEASE);
}

```

```

Hello, world

C:\Users\Анастасия\source\repos\lect
Нажмите любую клавишу, чтобы закрыть

```

4. Прогон программы, демонстрирующей выделение больших массивов памяти

```

#include <windows.h>
#include <stdio.h>
void main(void)
{
    PVOID pMem = NULL;
    int nPageSize = 4096;
    long SizeCommit = 0;
    int nPages = 200;
    SizeCommit = nPages * nPageSize;
    getchar();
    pMem = VirtualAlloc(0, SizeCommit,
        MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    if (pMem == NULL) printf("VirtualAlloc Error\n");
    getchar();
    pMem = VirtualAlloc(0, SizeCommit,
        MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    if (pMem == NULL) printf("VirtualAlloc Error\n");
    getchar();
    pMem = VirtualAlloc(0, SizeCommit,
        MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    if (pMem == NULL) printf("VirtualAlloc Error\n");
    getchar();
    pMem = VirtualAlloc(0, SizeCommit,
        MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    if (pMem == NULL) printf("VirtualAlloc Error\n");
    getchar();
}

```

```

Hello, world

C:\Users\Анастасия\source\repos\lect
Нажмите любую клавишу, чтобы закрыть

```

5. Прогон программы выделения памяти в стандартной куче

```

#include <windows.h>
#include <stdio.h>
void main(void)
{
    HANDLE hHeap;
    long Size = 1024;
    long Shift = 1017;
    char* pHeap;
    char* String;
    hHeap = GetProcessHeap();
    pHeap = (char*)HeapAlloc(hHeap, HEAP_ZERO_MEMORY, Size);
    if (pHeap == NULL) { printf("HeapAlloc error\n"); return; }
    String = pHeap + Shift;
    sprintf_s(String, 100, "Hello, world");
    printf("Heap contents string: %s\n", String);
    HeapFree(hHeap, 0, pHeap);
}

```

Hello, world

C:\Users\Анастасия\source\repos\lect
Нажмите любую клавишу, чтобы закрыть

6. Прогон программы, моделирующей обращение к сторожевым страницам

```

#include <windows.h>
#include <stdio.h>
void main(void)
{
    PVOID pMem = NULL;
    char* String;
    char* pMemCommitted;
    int nPageSize = 4096;
    int Shift = 4000;
    pMem = VirtualAlloc(0, nPageSize * 16,
        MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE |
        PAGE_GUARD);
    pMemCommitted = (char*)pMem + 3 * nPageSize;
    String = pMemCommitted + Shift;
    __try {
        sprintf_s(String, 100, "Hello, world");
        printf("Before exception number 0x80000001 string: %s \n", String);
    }
    __except (EXCEPTION_EXECUTE_HANDLER)
    {
        sprintf_s(String, 100, "Hello, world");
        printf("After exception number 0x80000001 string: %s \n", String);
    }
    VirtualFree(String, 0, MEM_RELEASE);
}

```

Hello, world

C:\Users\Анастасия\source\repos\lect
Нажмите любую клавишу, чтобы закрыть

7. Прогон программы, демонстрирующей отображение файла в память

```

#include <windows.h>
#include <stdio.h>
void main(void) {
    HANDLE hMapFile;
    LPVOID lpMapAddress;
    HANDLE hFile;
    char* String;
    hFile = CreateFile(L"MyFile.txt", // имя файла
        GENERIC_READ | GENERIC_WRITE, // режим доступа
        FILE_SHARE_READ | FILE_SHARE_WRITE, // совместный доступ
        NULL, // защита по умолчанию
        CREATE_ALWAYS, // способ создания
        FILE_ATTRIBUTE_NORMAL, // атрибуты файла
        NULL); // файл атрибутов
    if (hFile == INVALID_HANDLE_VALUE) printf("Could not open file\n");
    hMapFile = CreateFileMapping(hFile, NULL, PAGE_READWRITE, 0, 20, L"MyFileObject");
    if (hMapFile == NULL) {
        printf("Could not create file-mapping object.\n"); return;
    }
    lpMapAddress = MapViewOfFile(hMapFile, FILE_MAP_ALL_ACCESS, 0, 0, 0);
    if (lpMapAddress == NULL) {
        printf("Could not map view of file.\n"); return;
    }
    String = (char*)lpMapAddress;
    sprintf_s(String, 100, "Hello, world");
    printf("%s\n", String);
    if (!UnmapViewOfFile(lpMapAddress)) printf("Could not unmap view of file.\n");
}

```

Hello, world

C:\Users\Анастасия\source\repos\lect
Нажмите любую клавишу, чтобы закрыть